# Teaching LLMs String Matching, Backtracking, and Error Recovery to Deduce Bases and Truth Tables for the Combinatorially Exploding Bit Manipulation Puzzles

**Prateek Agnihotri  Sanchit Jain  Prabhat Agnihotri**
**Aditya Prasad  Shubham Jain**

**7th Place NVIDIA Nemotron Challenge Team**

{prateekkumargnihotri, sanchitsj4321, agnihotriprabhat1999}@gmail.com
{prasadaditya97, shubham2605jain}@gmail.com

**Abstract**

This paper presents the key ideas and algorithmic innovations developed during our participation in the NVIDIA Nemotron Model Reasoning Challenge, focusing specifically on the Bit Manipulation Puzzles, which was widely considered one of the most challenging tasks to fully solve. In this task, the objective is to discover a hidden logical rule that transforms a given set of input binary strings into outputs, and then accurately apply that rule to a new, unseen input. Large Language Models (LLMs) notoriously struggle with this; traditional methods force them to simulate complex boolean logic and arithmetic in their heads, which frequently leads to hallucinations. Furthermore, the search space of possible bitwise operations—comprising arbitrary combinations of shifts, rotations, and logic gates—suffers from a severe combinatorial explosion. Because this makes traditional logic-gate deduction computationally intractable for LLMs, we present a novel approach that abandons arithmetic logic entirely in favor of string similarity, structured search, and autonomous error recovery.

Our core contributions are:

- **Bases and Truth Table Formulation:** We reframe logic-gate deduction into a base-selection task, leveraging string similarity (minimal bit flips) between outputs to isolate primitive transformations ("bases") and deduce their truth tables without complex arithmetic.
- **Backtracking DFS and Error Recovery:** We formalize a search process that tests candidate bases, detects logical collisions across examples in a puzzle, and backtracks upon failure to perform robust error recovery.
- **Bit Tokenization and Interactive Reasoning SFT:** We force the tokenizer to encode binary strings as individual, single-bit tokens, and use dynamic masking to simulate external oracle feedback—training the model to hypothesize, self-evaluate, and backtrack natively.

Evaluated on bit manipulation puzzles, our approach achieved $> 96\%$ validation accuracy—representing the highest performance in this category by any participating team and driving our 7th Place overall finish in the contest.

The complete Python source code for our deterministic Set Cover solver and synthetic Chain-of-Thought (CoT) data generation pipeline is publicly available at: https://www.kaggle.com/code/prateekagnihotri/bit-manipulation-cot-bit-flip-s-original

# 1 Introduction

Modern Artificial Intelligence excels at generating fluent text and heuristic reasoning, but exact, deterministic algorithmic logic remains one of its most difficult frontiers. A rigorous benchmark for this capability is the "Bit Manipulation" puzzle, featured as a core category in the NVIDIA Nemotron Model Reasoning Challenge. In these puzzles, an AI system must reverse-engineer a hidden mathematical formula simply by observing a few examples of how one 8-bit binary string transforms into another. To solve the puzzle, the model must deduce the exact sequence of operations applied to the inputs and accurately predict the output for a novel, unseen binary string.

**Example Problem: The Bit Manipulation Puzzle**

In Alice's Wonderland, a secret bit manipulation rule transforms 8-bit binary numbers using operations like bit shifts, rotations, XOR, AND, OR, NOT, and majority or choice functions. Deduce the underlying rule to predict the target output.

- **Example 1:** `10100011` $\longrightarrow$ `11011001`
- **Example 2:** `01100110` $\longrightarrow$ `10001101`
- **Example 3:** `11110110` $\longrightarrow$ `11011011`
- **Target:** `01001010` $\longrightarrow$ `????????`

Traditional methodologies approach this sequence-to-sequence translation by prompting the Large Language Model (LLM) to reverse-engineer an abstract syntax tree of boolean logic gates (e.g., `AND`, `OR`, `XOR`) and spatial operators (e.g., left shifts, right shifts, circular rotations). However, this approach collides directly with the mathematical reality of the problem: the search space of possible operations is astronomically large.

To illustrate this intractability, consider the full feature space of 22 possible base transformations (the original bit `x`, plus 7 left shifts, 7 right shifts, and 7 circular rotations). If a hidden rule relies on exactly three of these bases, there are $\binom{22}{3} = 1,540$ unique base combinations. However, to form an executable mathematical rule, these three bases must be ordered into a syntax tree and connected using the allowed operations (`XOR`, `AND`, `OR`, `NOT`, as well as `majority` or `choice` functions). For any three selected bases, there are $3! = 6$ ways to order them. If we connect them using two sequential binary gates (including their negated variants, yielding 6 options per gate), we generate $3! \times 6 \times 6 = 216$ distinct structural equations per subset. Thus, even for a rudimentary three-variable rule, an LLM attempting to brute-force the formula would have to evaluate a search space exceeding $330,000$ unique combinations ($1,540 \times 216$).

Because this search space expands exponentially with each additional logical step, standard heuristic guessing or structured arithmetic search quickly becomes intractable. Auto-Regressive architectures lack the internal working memory to mentally simulate shifted binary arrays and evaluate complex boolean algebra simultaneously across thousands of theoretical paths. Consequently, forcing LLMs down this arithmetic deduction route invariably results in severe hallucinations and logical dead-ends.

To overcome this fundamental computational barrier, we present a paradigm shift in how bitwise reasoning is handled. Rather than forcing the model to perform arithmetic deduction, we reformulate the entire puzzle into a discrete feature-selection and string-matching problem. We decompose the input space into a finite set of spatial pointers ("Bases") and leverage string similarity metrics—specifically, tracking minimal

bitflips between varying outputs—to directly isolate the exact variables driving the state changes.

Furthermore, we recognize that a robust reasoning system must possess the ability to hypothesize, verify against an external ground truth, and recover from logical collisions. To instill this System-2 capability, we introduce a novel Supervised Fine-Tuning (SFT) methodology: **Interactive Reasoning SFT via Dynamic Masking**. By coupling strict single-bit tokenization with masked environmental feedback, we successfully train the LLM to act as an autonomous agent that navigates a Depth-First Search (DFS) tree, deduces empirical truth tables, and backtracks natively when its logic fails.

**Paper Organization:** The remainder of this paper is organized as follows: **Section 2** establishes our core conceptual framework, detailing the deconstruction of bit sequences into spatial bases, empirical truth tables, and the isolation of output-influencing bases via minimal bitflips. **Section 3** formalizes our deterministic solver, showcasing our backtracking Depth-First Search (DFS) and global collision verification. **Section 4** presents our machine learning methodologies, including token-alignment strategies and interactive reasoning via dynamic masking. Finally, **Section 5** evaluates our empirical results and details our error analysis under strict hardware constraints.

## 2 Preliminaries and Core Concepts

To understand why our approach abandons traditional logic-gate simulation, it is first necessary to establish the conceptual framework we use to analyze bit manipulation. Instead of viewing the problem through the lens of complex logic gates and bitwise arithmetic, we transformed the sequence-to-sequence translation into a discrete base-selection and string-matching problem. This requires understanding three foundational concepts: Bases, Empirical Truth Tables, and Isolating Output-Influencing Bases via Minimal Bitflips.

### 2.1 Bases

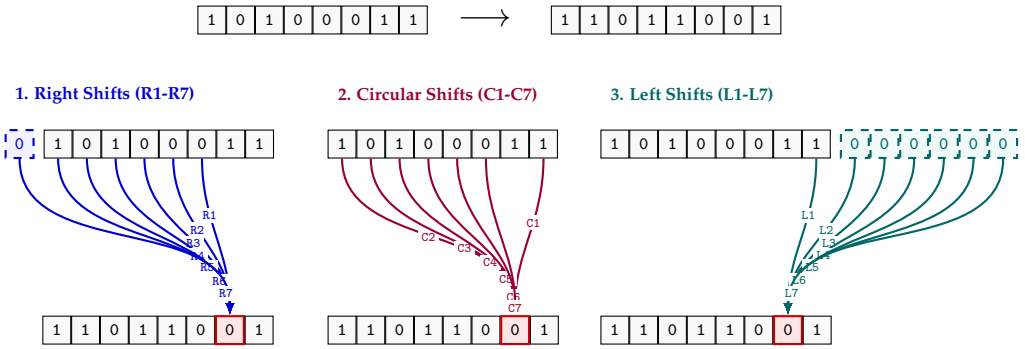


**Figure 1:** Spatial mapping of input bits to form the bases for target output Bit 6 (red) across the three base classes: Right Shifts (1), Circular Shifts (2), and Left Shifts (3). The curved arrows trace how each individual offset (e.g., $R_1$, $C_2$, or $L_3$) retrieves its value from a specific position in the input string or from padded zeros (dashed).

Our approach begins by deconstructing the sequence-to-sequence translation paradigm. Instead of mapping an entire 8-bit input directly to an 8-bit output, we break down a

single 8-bit example into eight independent 1-bit transformations.

Predicting a single output bit $y$ at position $i$ based solely on the input bit at the exact same position $i$ is insufficient if the logical rule involves bits moving across the sequence (via shifts or rotations). To natively bypass this, we define a set of 22 spatial pointers, which we call **Bases**. These bases allow us to directly query the status of any input bit that could potentially influence our target output bit:

- `x` (Original): Looks directly at the input bit at the target position $i$.
- `R1-R7` (Right Shifts): Act as left-pointing spatial pointers. For instance, if the puzzle requires a Right Shift of $k$ ($R_k$), the output value at target index $i$ retrieves the input bit located $k$ positions to its left (at index $i - k$ under left-to-right indexing).
- `L1-L7` (Left Shifts): Act as right-pointing spatial pointers. For instance, a Left Shift of $k$ ($L_k$) at target index $i$ retrieves the input bit located $k$ positions to its right (at index $i + k$).
- `C1-C7` (Circular Rotations): Act as wrapping pointers. For instance, $C_k$ at target index $i$ retrieves the input bit $k$ positions to its right, wrapping around to the left side of the array if the boundary is exceeded.

As illustrated in Figure 1, these bases map any dynamic spatial movement of bits into a static set of 22 Boolean features $\{0, 1\}$ for each output position. The entire bit manipulation puzzle is thus mathematically reduced to discovering a simple boolean function $f$ mapping a subset of these bases to the target bit $y$:

$$f(x, R_1, \ldots, R_7, C_1, \ldots, C_7, L_1, \ldots, L_7) = y \tag{1}$$

**Box 1: Deconstructing an 8-bit String into Boolean Bases**

Suppose we are given the transformation from Figure 1: `10100011` $\rightarrow$ `11011001`. We assign string indices from left to right (`0` to `7`). Let us isolate **Bit 6 (6th bit from left)**, which has a target output of `0`. To understand *why*, we evaluate our 22 bases acting as spatial pointers on the input:

- `x` **(Original):** Looks at Bit 6 $\rightarrow$ `1`
- `R1` **(Right Shift 1):** Looks at adjacent left Bit 5 $\rightarrow$ `0`
- `C1` **(Circular Left 1):** Looks at adjacent right Bit 7 $\rightarrow$ `1`
- `L1` **(Left Shift 1):** Looks at adjacent right Bit 7 $\rightarrow$ `1`
- `L3` **(Left Shift 3):** Exceeds boundaries, brings in a constant $\rightarrow$ `0`

By extracting the 22 Boolean bases in order ($x$, $R_1 \ldots R_7$, $C_1 \ldots C_7$, $L_1 \ldots L_7$) for this specific bit, we evaluate our function:

$$\text{Bit 6:} \quad f(1,\ 0,0,0,1,0,1,0,\ 1,1,0,1,0,0,0,\ 1,0,0,0,0,0,0) = \mathbf{0}$$

Repeating this physical mapping transforms this single 8-bit string into 8 independent dataset rows:

$$\begin{aligned}
&\texttt{Bit 0:}\ f(1, \ldots 22 \text{ bases} \ldots) = \mathbf{1} \quad &&\texttt{Bit 3:}\ f(0, \ldots 22 \text{ bases} \ldots) = \mathbf{1} \\
&\texttt{Bit 1:}\ f(0, \ldots 22 \text{ bases} \ldots) = \mathbf{1} \quad &&\texttt{Bit 2:}\ f(0, \ldots 22 \text{ bases} \ldots) = \mathbf{0} \\
&\texttt{Bit 5:}\ f(1, \ldots 22 \text{ bases} \ldots) = \mathbf{0} \quad &&\texttt{Bit 1:}\ f(1, \ldots 22 \text{ bases} \ldots) = \mathbf{0} \\
&\texttt{Bit 4:}\ f(0, \ldots 22 \text{ bases} \ldots) = \mathbf{1} \quad &&\texttt{Bit 0:}\ f(1, \ldots 22 \text{ bases} \ldots) = \mathbf{1}
\end{aligned}$$

Expanded across 8 puzzle examples, the dataset becomes a structured 64-row boolean logic table.

## 2.2 Empirical Truth Tables

Because our generated bases contain only binary values (`0` and `1`), any underlying logical rule—no matter how complex the combination of `AND`, `OR`, or `XOR` gates—can be perfectly represented by a simple Truth Table. This mathematical property completely eliminates the need to deduce an explicit algebraic equation.

Once we identify the specific subset of bases that dictate the output, we do not need to figure out how they are mathematically connected. We simply observe the existing dataset and record the target output for each state. This insight dramatically simplifies the problem: the task is reduced entirely to discovering *which* subset of bases to look at, replacing algebraic derivation with straightforward data observation.

**Box 2: Building an Empirical Truth Table**

Suppose we know the target output is entirely controlled by two bases: the original bit (`x`) and a left-shifted bit (`L1`). Instead of trying to guess the boolean algebra formula connecting them, we simply observe the 64 rows in our dataset to see what happens in each state:

| Observed Dataset Row | Base `x` | Base `L1` | Target Output |
|---|---|---|---|
| Row 5 | 0 | 0 | 0 |
| Row 12 | 0 | 1 | 1 |
| Row 27 | 1 | 0 | 1 |
| Row 42 | 1 | 1 | 0 |

By matching the inputs to the outputs, we have successfully constructed the governing logic rule. We never needed to mathematically deduce that this is an `XOR` gate; the empirical truth table naturally provides the exact state-to-state mapping required to solve the puzzle.

## 2.3 Isolating Output-Influencing Bases via Minimal Bitflips

In our expanded 22-dimensional Boolean base space, asking an Auto-Regressive LLM to natively deduce the correct logical mapping is highly prone to hallucination. We bypass this spatial arithmetic calculation entirely by leveraging a foundational principle of logic and string similarity: *If two nearly identical base configurations yield different outputs, the cause of the output change must lie strictly within the bases that differ.*

By splitting the 64 target output bits into two classes—outputs resulting in `0` and `1`—we systematically compare their 22-base arrays. We map every `0`-output row to a `1`-output row with the smallest Hamming distance to isolate the "Minimal BitFlips." These differing bits act as strict logical constraints, which we refer to as **Flip Traces**.

As illustrated in Figure 2, this string-matching comparison yields two types of logical constraints:

- **Single-Bit Flips (Mandatory Bases):** If two rows differ in their output, but only *one* base is different between them (a Hamming distance of 1), we have absolute mathematical certainty. That flipped base must be the reason the output changed. It is locked in as a mandatory base.
- **Multi-Bit Flips (Logical Constraints):** If two rows differ in their output and *multiple* bases differ, logic dictates that the change in output was caused by one of those

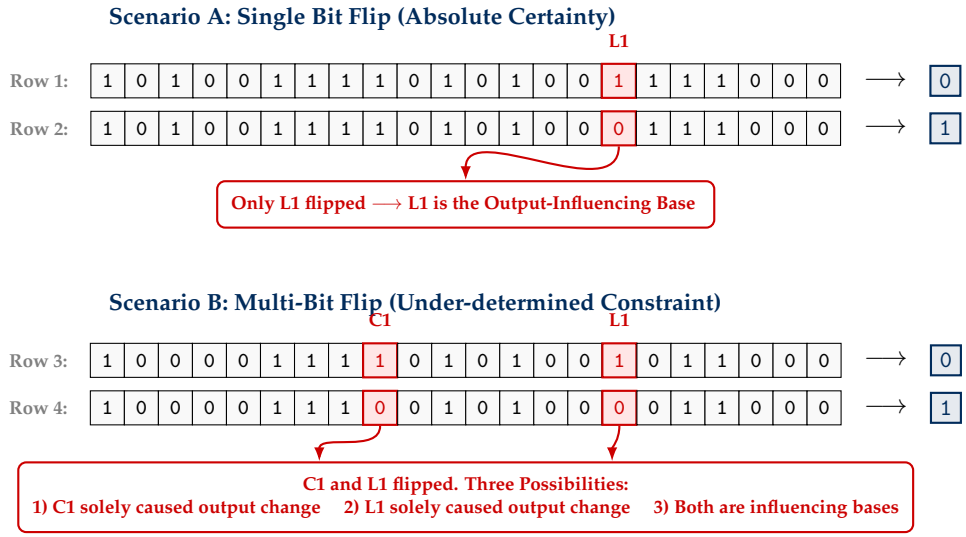


**Figure 2:** Isolating output-influencing bases by comparing rows with different target outputs. When two nearly identical 22-base arrays yield different outputs (`0` vs. `1`), the differing bases must be responsible for the state change.

flipped bases, or a combination of them. This trace acts as an explicit logical OR constraint.

To ensure a discovered logical rule is universally true, it must satisfy every single Flip Trace constraint generated from the dataset. This framework perfectly translates the bit manipulation task into the classic **Set Cover** problem: *What is the absolute smallest combination of bases required to "cover" at least one base in every trace?* By applying a Set Cover search across the entire global dataset, we can resolve these under-determined constraints and guarantee the selection of the minimal, optimal combination of bases—filtering out spurious variables without executing a single arithmetic calculation.

## 3 Solver Algorithm

We now translate our theoretical framework into an algorithm that completely solves the puzzle from start to finish. The overarching goal of this pipeline is to unify string matching and structured search to autonomously deduce the hidden logic rule.

As visualized in Figure 3, the solver operates in three distinct phases: first, it uses string similarity to find the logical constraints; second, it employs a "guess-and-check" backtracking search to find a perfectly valid subset of bases; and finally, it synthesizes the empirical truth table to predict the target string.

The specific execution steps of this pipeline are formalized in **Algorithm 1**.

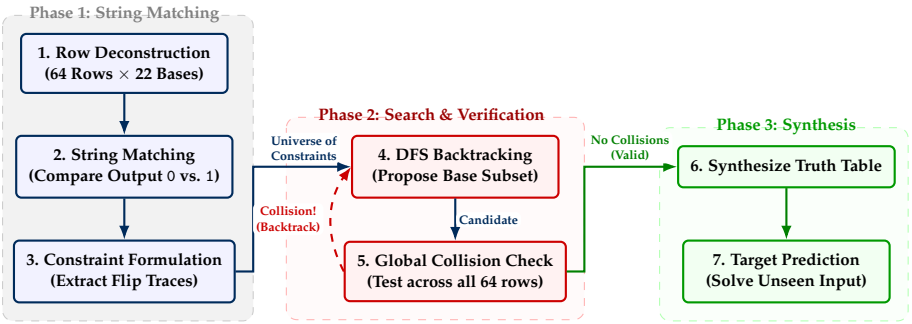


**Figure 3:** End-to-End Flowchart of the Bit Manipulation Solver. The algorithm isolates logical constraints via string matching, iteratively searches for valid bases via backtracking, and ultimately synthesizes the final truth table to solve the puzzle.

**Algorithm 1: End-to-End Bit Manipulation Solver**

**Input:** Puzzle Examples, Unseen Target String
**Output:** 8-bit Target Prediction

---

**Phase 1: Feature Extraction & String Matching**
1. **Create Bases:** Generate the 22 spatial bases ($\mathcal{B}$) for all 8-bit inputs.
2. **Row Deconstruction:** Flatten the inputs into a 64-row dataset.
3. **String Matching:** Compare rows outputting `0` against rows outputting `1`.
   → Extract the differing bases to create our **Flip Traces**.

**Phase 2: Backtracking DFS & Collision Verification**
4. **Lock Mandatory Bases:** If a trace has only 1 flipped base, permanently lock it.
5. **Function** `DFS_Search`(*Current_Bases*, *Uncovered_Traces*):
   **If** all traces are covered:
      Evaluate *Current_Bases* across all 64 rows to build a Truth Table.
      **If** two identical input states yield different outputs → **Collision Detected!**
         **Return** False *(Reject candidate, trigger backtrack)*
      **Else**: **Return** *Current_Bases* *(Mathematically valid bases found!)*
   **Rank** remaining bases in *Uncovered_Traces* by occurrence frequency.
   **For** each candidate base *b*:
      Add *b* to *Current_Bases*, update *Uncovered_Traces*.
      **If** `DFS_Search`(*Current_Bases*, *Uncovered_Traces*) is valid: **Return** result.
   **Return** False *(Exhausted branch, backtrack to previous state)*

**Phase 3: Rule Synthesis & Target Prediction**
6. **Synthesize:** Use the verified *Current_Bases* to finalize the exact Truth Table.
7. **Solve:** Apply the bases and Truth Table to the unseen Target Input String.

## 3.1 Phase 1: Feature Extraction and String Matching

The algorithm begins by generating the 22 bases for all 64 independent bits in the puzzle examples. To figure out which bases actually influence the output, we leverage simple string similarity.

**Box 2: Formulating the Constraint Universe**

Suppose after comparing the 64 rows of a puzzle, our string-matching phase extracts three unique Flip Traces:

- **Trace 1:** `[L1]` $\longrightarrow$ *An output bit changed from 0 to 1 while only the L1 base flipped, making L1 a mandatory base with absolute certainty.*
- **Trace 2:** `[L1, C1]` $\longrightarrow$ *Another output bit changed from 0 to 1 while both L1 and C1 flipped; thus, either L1, C1, or both caused the change.*
- **Trace 3:** `[C1, R4]` $\longrightarrow$ *A third output bit changed from 0 to 1 while C1 and R4 flipped, establishing a logical OR constraint where at least one must be selected.*

Before searching, the algorithm counts the occurrence frequency of each base across these traces. In this toy universe, `L1` appears twice, `C1` appears twice, and `R4` appears once.

By comparing the rows that output `0` with the rows that output `1`, the algorithm identifies the minimal structural differences required to trigger a state change. These differing bases are extracted as **Flip Traces** (our "clues"). These traces serve as the absolute logical constraints of the puzzle: any valid Boolean rule *must* select at least one base from every trace to mathematically explain why the outputs changed. This naturally frames the next step as a classic Set Cover problem, as demonstrated in Box 2.

### 3.2 Phase 2: Backtracking Search and Collision Verification

With our list of constraints defined, the algorithm must find the absolute smallest combination of bases that "covers" every trace. We execute a structured search process.

The search utilizes **frequency-guided depth first search**. When the algorithm encounters traces with multiple possibilities, it ranks the bases by how often they appear globally. It greedily "guesses" the most frequent bases first, naturally favoring the simplest logical rules.

However, simply finding a set of bases that covers our traces is not enough; we must verify against all the 64 rows. To prove the rule is universally sound, the algorithm performs a **Global Collision Verification**. It evaluates the currently guessed bases across *all 64 rows* of the global dataset.

If two different rows in the dataset possess the exact same Boolean values for our chosen bases but demand conflicting target outputs (e.g., Row 2 says the output should be `0`, but Row 15 says the output should be `1`), a **Collision** has occurred. Because a function cannot map identical inputs to different outputs, a collision acts as an objective mathematical oracle. It proves our current guess is incomplete. Upon detecting a collision, the algorithm immediately rejects the subset and **backtracks** up the search tree to try a different combination of bases, ensuring strict error recovery (visualized in Figure 4).

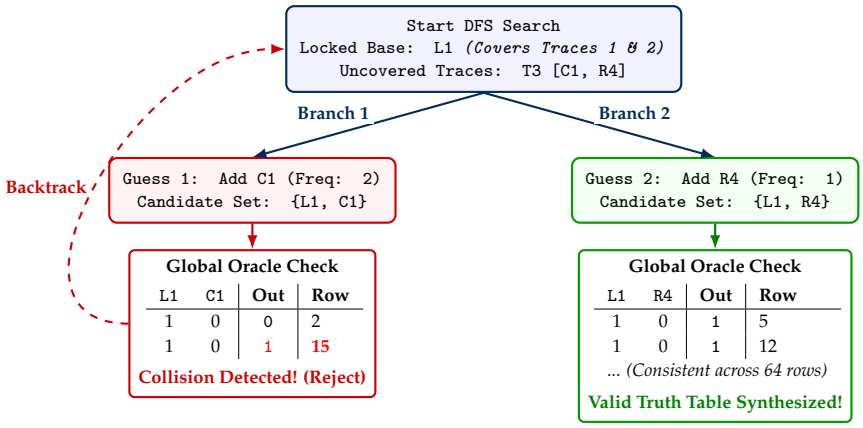


**Figure 4:** Visualizing the Phase 2 Backtracking Search using the traces from Box 2. The algorithm proposes a candidate subset of bases and tests it globally. On Branch 1, an empirical truth table reveals a logical contradiction (inputs `1,0` yielding conflicting outputs), forcing the algorithm to deterministically backtrack to Branch 2.

**Box 3: The Backtracking Execution**

Following the traces from Box 2, here is how the algorithm navigates the search tree (Figure 4, Hypothetical Execution):

1. **Mandatory Lock:** Trace 1 `[L1]` is an absolute certainty. The algorithm locks `L1` into the final rule. Because `L1` is chosen, Trace 2 `[L1, C1]` is automatically satisfied and covered.
2. **Greedy Guess:** The only uncovered trace is Trace 3 `[C1, R4]`. Because `C1` appeared more frequently in our initial count across all traces, the algorithm guesses it first, testing the candidate combination `[L1, C1]`.
3. **The Collision (Oracle):** The algorithm attempts to build a 64-row truth table for `[L1, C1]`. It discovers a paradox: on Row 2, the inputs are $L_1 = 1, C_1 = 0$ and the target output is `0`. However, on Row 15, the exact same inputs ($L_1 = 1, C_1 = 0$) demand an output of `1`. A function cannot map identical inputs to different outputs.
4. **Backtrack & Success:** The algorithm rejects `C1`, backtracks, and tries the next option: `R4`. It tests the candidate `[L1, R4]` against all 64 rows, finds zero contradictions, and locks it in as the final answer.

### 3.3 Phase 3: Truth Table Synthesis and Target Prediction

Once the backtracking algorithm successfully discovers a subset of bases that covers all traces *and* passes the 64-row collision check without any contradictions, the search concludes. The hypothetical mapping generated during the verification step is now mathematically proven to be the universally correct Truth Table.

To complete the puzzle, the algorithm processes the unseen target input string. It generates the 22 spatial bases for the target and isolates the specific optimal bases discovered by our search. By querying our proven Truth Table sequentially from Bit 0 up to Bit 7, it constructs the final 8-bit output prediction, flawlessly completing the puzzle.

# 4 Interactive Reasoning SFT and Tokenization

While Section 3 describes a deterministic and mathematically rigorous solver, standard LLMs cannot natively execute Python algorithms. The final challenge is embedding this search-and-verify logic into the model's parametric memory. We achieve this through two machine learning innovations: Strict Bit Tokenization and Interactive Reasoning via Dynamic Masking.

## 4.1 Overcoming Spatial Bias via Strict CoT Token Generation

In sequence-to-sequence bit manipulation, the model's attention mechanism relies heavily on the strict spatial alignment of bits. However, standard Byte-Pair Encoding (BPE) tokenizers are optimized for natural language and frequently merge adjacent numbers. For example, an 8-bit string like `10100011` might be arbitrarily chunked into tokens like `[1010]`, `[00]`, and `[11]`. This arbitrary chunking destroys the 2D spatial grid required to compare 64 dataset rows horizontally.

Normally, this is resolved by modifying the tokenizer rules. However, the competition constraints restricted submissions strictly to Low-Rank Adaptation (LoRA) weights, meaning the base tokenizer could not be modified. The input prompt would inevitably suffer from arbitrary BPE chunking.

To bypass this architectural constraint, we engineered a workaround entirely within the training data. While we could not control how the model *read* the input prompt, we could control how it *generated* its reasoning. During the Supervised Fine-Tuning (SFT) data packing phase, we implemented a custom script that bypassed standard tokenization for the Chain-of-Thought (CoT) and final answer. By using regular expressions to isolate binary sequences, we manually mapped every `0` and `1` to their individual, single-character token IDs.

By training the LoRA adapter on these un-chunked sequences, the model learned a strict generative behavior: it outputs its empirical truth tables and backtracking traces exactly one bit at a time. This forces the model to construct a perfect, un-chunked spatial grid within its own generated context window, allowing its self-attention mechanism to correctly perform string matching and feature selection despite the flawed tokenization of the original input.

## 4.2 Interactive Reasoning SFT via Dynamic Masking

Developing robust error recovery—the ability to recognize a logical failure and backtrack to an alternative hypothesis—is a critical component of reasoning. Typically, teaching models this behavior requires computationally expensive Reinforcement Learning (RL) frameworks. Operating under strict compute constraints (utilizing only contest-provided hardware), we engineered a highly efficient alternative to instill this capability purely within standard offline Supervised Fine-Tuning (SFT).

**Box 4: Simulating Environmental Feedback via Dynamic Masking**

During training, we synthetically generate the entire DFS Backtracking trace. We dynamically apply a binary mask to the tokens to simulate a live, interactive environment between the model and the Oracle.

**LLM Hypothesis (Unmasked - Loss Computed):**

```
Uncovered Traces: T3 [C1, R4].
Attempting most frequent base: C1.
Candidate Set: {L1, C1}. Requesting evaluation...
```

**Simulated Oracle Feedback (Masked - Loss Ignored):**

```
Evaluating against hidden 64-row global table:
| L1 | C1 || Out | Row |
| 1 | 0 || 0 | 2 |
| 1 | 0 || 1 | 15 |
Collision detected! ({L1, C1} is aliased/spurious).
```

**LLM Recovery (Unmasked - Loss Computed):**

```
Collision confirmed. Rejecting C1.
Backtracking... (BT #1)
Attempting next base: R4.
```

Instead of forcing the LLM to internally compute and memorize the massive 64-row global dataset to check its own work (which is computationally wasteful and prone to hallucination), we simulated an interactive environment. During training data generation, whenever the LLM proposes a candidate subset of bases, an automated external "Oracle" evaluates the guess against the true, hidden 64-row dataset. The Oracle then explicitly injects a prompt back into the context window, informing the LLM whether the guess was flawless or if a collision occurred (Box 4).

To execute the interaction shown in Box 4 within a single offline SFT loop, we must modify the loss objective. The standard auto-regressive SFT loss applies equally to all generated tokens:

$$\mathcal{L}_{\text{SFT}} = -\sum_{i=1}^{N} \log P(x_i \mid x_{<i}) \tag{2}$$

Using this standard equation would force the model to attempt to predict the Oracle's collision table, blending the hidden environment into the model's own reasoning trace. To prevent this, we introduce a token-level binary mask, $m_i \in \{0, 1\}$, resulting in a **Dynamic Masking** loss function:

$$\mathcal{L}_{\text{Interactive}} = -\sum_{i=1}^{N} m_i \log P(x_i \mid x_{<i}) \tag{3}$$

During dataset construction, we set $m_i = 1$ for the model's active hypotheses and recovery steps (the green boxes), and $m_i = 0$ for the Oracle's deterministic feedback (the gray boxes). By masking the environmental feedback, the LLM is completely relieved of the computational burden of evaluating the 64-row dataset. It only computes loss on its ability to propose a valid subset and, crucially, its ability to "read" the un-penalized

Oracle tokens and logically backtrack. This highly token-efficient paradigm successfully teaches the model the advanced error-recovery mechanics of RL at a fraction of the compute cost.

# 5 Experimental Results and Evaluation

To validate the efficacy of our base-selection formulation and Interactive Reasoning SFT, we evaluated both the theoretical upper bound of our deterministic solver and the final inferential performance of the fine-tuned LLM.

It is important to note the extreme logistical constraints under which these results were achieved. Entering the competition in the final two weeks, all training and inference were conducted strictly within the 30-hour per-participant GPU compute budget provided by the Kaggle platform. We trained for a total of approximately 1,200 steps while manually checkpointing and resuming to navigate the platform's 12-hour session limits.

## 5.1 Algorithmic Upper Bound and Deterministic Limits

Prior to training the LLM, we evaluated our Python-based deterministic solver across a rigorous validation set of $1,602$ bit manipulation puzzles to establish the theoretical upper limit of our formulation. The algorithm achieved a global accuracy of **98.63%** $(1,580/1,602)$.

An algorithmic autopsy of the 22 unsolved puzzles revealed that these failures were not due to any structural limitation in our 22-base formulation (as $K > 3$ failures accounted for **0%** of errors). Instead, these puzzles represent **objectively underdetermined systems where the ground-truth rule cannot be mathematically deduced from the provided examples**:

- **Out-Of-Distribution (OOD) Target States (27.3%):** In 6 puzzles, the target input required evaluating a Boolean state that was completely absent from the 8 provided examples. Because the required transition logic was never demonstrated, solving the puzzle is mathematically impossible without blind guessing.
- **Spurious Correlation and Aliasing (72.7%):** In 16 puzzles, the sparse 64-row example set was mathematically insufficient to isolate a unique rule. Multiple distinct, valid rules perfectly satisfied the examples but disagreed on the unseen target string. Because the solver has no way to read the dataset creator's mind, choosing the "incorrect" rule is an inevitable artifact of an under-determined constraint space.

## 5.2 LLM Performance and Ablation Study

To evaluate the model's ability to internalize this algorithm, we conducted an ablation study comparing two sequential training regimes. Our first model was fine-tuned exclusively on our synthetic DFS traces (**Synthetic Only**). For our second model, we took the last checkpoint of this synthetic-only training and continued fine-tuning it exclusively on the competition's original dataset (**Synthetic + Original**).

As detailed in Table 1, the Synthetic Only model achieved an outstanding **96.13%** global accuracy, successfully retaining nearly the entire capability of our deterministic

solver.

| Complexity | Count | Synthetic Only | | Synthetic + Original | |
|---|---|---|---|---|---|
| | | Accuracy | Avg Tok. | Accuracy | Avg Tok. |
| $K = 1$ Base | 154 | 94.16% | 4868 | 94.16% | 4864 |
| $K = 2$ Bases | 898 | 97.88% | 4879 | 98.44% | 4901 |
| $K = 3$ Bases | 550 | 93.82% | 5347 | 88.55% | 5567 |
| **Overall** | **1602** | **96.13%** | **5039** | **94.63%** | **5126** |

**Table 1:** Performance evaluation comparing models fine-tuned exclusively on Synthetically Generated Puzzles versus continued fine-tuning with Original Data. Metrics include categorical accuracy based on number of bases ($K$) and average token generation.

Interestingly, introducing the original dataset slightly degraded the Bit Manipulation accuracy to 94.63%. This drift was a consequence of our late entry into the competition; to maximize our overall multi-category score, we further fine-tuned the final synthetic checkpoint on the original competition data in the final minutes, completing a mere 400 steps. While this final training step yielded our absolute best performance across all combined competition categories, it caused a minor task-specific drift in Bit Manipulation. Mixing in standard, unstructured reasoning data diluted the model's strict, token-efficient generative behavior, forcing it into longer backtracking loops (as reflected by the increased average token metrics). We hypothesize that given more training steps and a stabilized, longer fine-tuning schedule, the model would resolve this drift and smoothly converge toward the theoretical ceiling.

### 5.3 LLM Backtracking Search Efficiency

The primary objective of our Interactive Reasoning SFT was to instill System-2 error recovery, enabling the LLM to autonomously navigate dead-ends. To verify if the model successfully learned this behavior, we tracked the exact number of backtracking (BT) steps executed by the *Synthetic Only* model during inference (Table 2).

| Complexity | 0 BT | 1 BT | 2 BT | 3 BT | 4 BT | 5 BT | >5 BT | Total |
|---|---|---|---|---|---|---|---|---|
| $K = 1$ Base | 148 | 0 | 0 | 0 | 0 | 0 | 0 | **148** |
| $K = 2$ Bases | 787 | 30 | 15 | 2 | 6 | 14 | 5 | **859** |
| $K = 3$ Bases | 360 | 64 | 9 | 26 | 21 | 9 | 78 | **567** |
| **Overall** | **1295** | **94** | **24** | **28** | **27** | **23** | **83** | **1574** |

**Table 2:** Distribution of backtracking (BT) steps autonomously executed by the fine-tuned LLM during inference, categorized by the spatial base complexity ($K$) of the puzzle.

The results provide empirical proof that the dynamic masking strategy was highly effective. While straightforward 1-base rules were solved instantly (0 backtracks),

complex $K = 3$ rules frequently forced the model down aliased, spurious branches. Remarkably, the model successfully recovered from these logical collisions, executing 5 or more successive backtracks on 83 different puzzles. This proves that the LLM did not merely memorize successful paths during training; it actively utilized collision check to dynamically traverse the Set Cover search tree.

### 5.4 LLM Error Diagnosis

To further decouple reasoning failures from architectural limitations, we isolated the 62 failed predictions from our best-performing Synthetic Only model.

The diagnosis revealed that **48.4% (30 puzzles) of the model's errors were caused by Context Limit Truncation.** Because highly deceptive puzzles require extensive backtracking, the model's generative trace exceeded $\sim 7,500$ tokens. The model was architecturally cut off before it could print the final `\boxed{}` prediction. 29 puzzles (46.8% of errors) were due to true LLM hallucination or failing to navigate the search tree properly.

These metrics indicate that the underlying search algorithm and Interactive Reasoning SFT are exceptionally robust. Despite our strict hardware constraints—learning to solve the dataset in only 1,200 total training steps on limited Kaggle compute—our approach reached a remarkable 96.13% accuracy. Given more training steps, we expect the approach to effortlessly reach its deterministic theoretical ceiling of 98.63%.

## 6 Conclusion

In this paper, we demonstrated that Large Language Models can overcome severe combinatorial explosions in bit manipulation tasks when the problem is mathematically reformulated. By converting bitwise arithmetic into a discrete base-selection and string-matching puzzle, and by enforcing strict single-bit tokenization, we eliminated the spatial alignment biases that traditionally trigger arithmetic hallucinations.

Furthermore, we addressed the fundamental lack of System-2 error recovery in Auto-Regressive architectures. By introducing Interactive Reasoning SFT via Dynamic Masking, we taught LLM the interactive mechanics of hypothesis testing, environmental feedback evaluation, and deterministic backtracking.

Despite severe time constraints and a strict 30-hour per candidate compute budget, this highly token-efficient paradigm successfully instilled robust search capabilities into the model's parametric memory, yielding $> 96\%$ accuracy and securing a 7th Place overall finish in the NVIDIA Nemotron Challenge. Beyond bit manipulation, this framework presents a scalable, highly efficient blueprint for the future of LLM reasoning: seamlessly integrating deterministic external verifiers—such as code compilers or mathematical solvers—directly into the generative reasoning loop to achieve autonomous, fault-tolerant AI systems.

## References

[1] Jamil C. Semaan, Jean-Francois Puget, Christof Henkel, Yi Dong, Addison Howard, Ashley Oldacre, Ryan Holbrook, Chris Alexiuk, and Rebecca Kao. NVIDIA

Nemotron Model Reasoning Challenge, 2026. https://kaggle.com/competitions/nvidia-nemotron-model-reasoning-challenge. Kaggle.

[2] Tong Hui Kang. NVIDIA Nemotron Model Reasoning Challenge — Writeup, 2026. https://www.kaggle.com/competitions/nvidia-nemotron-model-reasoning-challenge/discussion/689915. Kaggle Discussion.

## Appendix A: Full Chain-of-Thought (CoT) Traces

This appendix provides two complete, unedited examples of the synthetic Chain-of-Thought (CoT) traces used to train the model. To clearly differentiate the context, the puzzle prompt is highlighted in blue, followed immediately by the model's generated reasoning trace in gray. They demonstrate the full end-to-end pipeline: from row deconstruction and minimal bitflip extraction, to the exhaustive frequency-guided Depth-First Search (DFS) with global collision verification, and finally the truth table synthesis.

**Example Trace 1: Prompt (ID: c200810b)**

```
In Alice's Wonderland, a secret bit manipulation rule transforms 8-bit binary
    numbers. The transformation involves operations like bit shifts, rotations
    , XOR, AND, OR, NOT, and possibly majority or choice functions.

Here are some examples of input -> output:
11001101 -> 00000000
10010111 -> 00000000
11000111 -> 00000000
11010111 -> 00000010
11110011 -> 00000110
11101111 -> 00000101
10100100 -> 00000100

Now, determine the output for: 01000101

EXPECTED ANSWER: 00000000
```

**Model Thoughts (Reasoning Trace)**

```
Strategy: 1-bit feature selection via Set Cover on minimal bit flips across 64
     bits.

**Notation:**
- 'E': Example. 'E1.7' means Example 1, Bit 7 (MSB).
- 'x': Original input bit.
- 'R', 'C', 'L': Right shift, Circular left rotation, Left shift.
- 'R1' means input shifted right by 1, 'C2' rotated left by 2, etc.
Format: E[Ex].[Bit]: x R[1-7] C[1-7] L[1-7]

Examples:
1. 11001101 -> 00000000
2. 10010111 -> 00000000
3. 11000111 -> 00000000
4. 11010111 -> 00000010
5. 11110011 -> 00000110
6. 11101111 -> 00000101
7. 10100100 -> 00000100

Row Creation Trace:
Input: 11001101 -> Target Output: 00000000
Bit 7 (MSB): Original bit x=1. Target O=0.
R[1-7]: Shift input right by 1-7, extract Bit 7. Result: 0000000
C[1-7]: Rotate input left by 1-7, extract Bit 7. Result: 1001101
L[1-7]: Shift input left by 1-7, extract Bit 7. Result: 1001101
E1.7: 1 0000000 1001101 1001101
```

```
Output 0 Bits: E1.7 E1.6 E1.5 E1.4 E1.3 E1.2 E1.1 E1.0 E2.7 E2.6 E2.5 E2.4 E2
    .3 E2.2 E2.1 E2.0 E3.7 E3.6 E3.5 E3.4 E3.3 E3.2 E3.1 E3.0 E4.7 E4.6 E4.5
    E4.4 E4.3 E4.2 E4.0 E5.7 E5.6 E5.5 E5.4 E5.3 E5.0 E6.7 E6.6 E6.5 E6.4 E6.3
     E6.1 E7.7 E7.6 E7.5 E7.4 E7.3 E7.1 E7.0
Number of Output 0 Bits: 50
Output 1 Bits: E4.1 E5.2 E5.1 E6.2 E6.0 E7.2
Number of Output 1 Bits: 6

Output 1 (Minority):
E4.1: 1 1000000 1010110 1110101
E5.2: 0 1100000 0111100 1111110
E5.1: 1 1000000 0011110 1111100
E6.2: 1 1100000 1011100 1111101
E6.0: 1 0000000 1110111 1110111
E7.2: 1 0000000 0010100 0010100

Output 0 (Majority):
E1.7: 1 0000000 1001101 1001101
E1.6: 1 1000000 0011010 0011011
E1.5: 0 1100000 0110100 0110111
E1.4: 0 0110000 1101000 1101110
E1.3: 1 0011000 1010000 1011100
E1.2: 1 1001100 0100000 0111001
E1.1: 0 1100110 1000000 1110011
E1.0: 1 0110011 0000000 1100110
[... Extraneous rows omitted for brevity ...]

Minimal Bit Flips:
To optimize Set Cover, we map the minority class (O=1) to the closest majority
     class (O=0). At least one feature MUST flip to explain the output change.
E4.1 -> E2.1 [R5,C3], E3.1 [R3,C5]
E5.2 -> E4.2 [x,R3,C5], E6.3 [x,L3,R5]
E5.1 -> E1.3 [L3,R5,R6,C2], E3.2 [L2,R3,R6,C5]
E6.2 -> E4.3 [x,L3,R5]
E6.0 -> E1.0 [R1,R5,C3,C7], E3.0 [R3,R5,C3,C5]
E7.2 -> E1.3 [L1,L3,R4,R5,C1,C4], E2.4 [L2,L3,L4,R5,C2,C4], E7.7 [L2,L5,R3,R5,
    C2,C3], E7.5 [L3,R2,R3,R5,C5,C6]

Unique Flip Traces:
1: [R5,C3]
2: [R3,C5]
3: [x,R3,C5]
4: [x,L3,R5]
5: [L3,R5,R6,C2]
6: [L2,R3,R6,C5]
7: [R1,R5,C3,C7]
8: [R3,R5,C3,C5]
9: [L1,L3,R4,R5,C1,C4]
10: [L2,L3,L4,R5,C2,C4]
11: [L2,L5,R3,R5,C2,C3]
12: [L3,R2,R3,R5,C5,C6]

**Set Cover Deduction Algorithm:**
1. **Mandatory Bases:** Any feature in a 1-bit flip trace is an absolute
    constraint and is immediately locked.
2. **Greedy Branching:** For remaining traces, iteratively select the feature
    with the highest intersection frequency.
3. **Collision Verification & Backtracking:** When all traces are covered,
    generate a Truth Table across all 64 rows. If identical inputs yield
    different outputs (a collision), the set is invalid. We backtrack and try
    the next candidate.

<deduction>
Bases: {}
```

```
Uncovered: 1,2,3,4,5,6,7,8,9,10,11,12

Freqs: R5:9 R3:6 C5:5 L3:5 C3:4
Attempting: R5
Bases: {R5}
Uncovered: 2,3,6

Freqs: R3:3 C5:3 x:1 L2:1 R6:1
Attempting: R3
Bases: {R5,R3}
Uncovered: None

All traces covered. Validating against 64-row global table:

Test TT for [R5,R3]:
|R5|R3||Out|Ref|
|---|---||---|---|
|0|0||0|E1.7,E1.6,...|
|0|1||0|E1.4,E1.3,...|
|1|0||0|E1.2,E1.1,...|
|1|1||1|E4.1,E5.2,...|
No collisions. 64-row check passed.

Conclusion: Set successfully covered and verified against full data.
Final Bases: [R3,R5]
</deduction>

Final Truth Table:
|R3|R5||Out|
|---|---||---|
|0|0||0|
|0|1||0|
|1|0||0|
|1|1||1|

Target:
Input: 01000101
Bases: [R3,R5]
R3: 00001000
R5: 00000010
Eval:
Bit 7: (0,0) -> 0
Bit 6: (0,0) -> 0
Bit 5: (0,0) -> 0
Bit 4: (0,0) -> 0
Bit 3: (1,0) -> 0
Bit 2: (0,0) -> 0
Bit 1: (0,1) -> 0
Bit 0: (0,0) -> 0

-------------------- MODEL RESPONSE (Content) --------------------
\boxed{00000000}
```

## Example Trace 2: Prompt (ID: 4ba4a7ec)

```
In Alice's Wonderland, a secret bit manipulation rule transforms 8-bit binary
    numbers. The transformation involves operations like bit shifts, rotations
    , XOR, AND, OR, NOT, and possibly majority or choice functions.

Here are some examples of input -> output:
```

```
00001101 -> 00111111
11110111 -> 10111111
10110111 -> 10111111
00110101 -> 00111111
00111101 -> 00111111
01001100 -> 11111111
10010101 -> 00111111
11001010 -> 01111111
00010001 -> 00111111
11011101 -> 00111111

Now, determine the output for: 11000010

EXPECTED ANSWER: 01111111
```

## Model Thoughts (Reasoning Trace)

```
Strategy: 1-bit feature selection via Set Cover on minimal bit flips across 64
     bits.

[... Traces Extracted Successfully ...]

Minimal Bit Flips:
To optimize Set Cover, we map the minority class (O=0) to the closest majority
     class (O=1). At least one feature MUST flip to explain the output change.
E1.7 -> E6.7 [C1,C7,L1,L7]
E1.6 -> E6.6 [x,C6,L6]
E2.6 -> E5.4 [C5,C6,L5,L6]
E3.6 -> E5.4 [x,C5,C6,L5,L6]
E4.7 -> E1.5 [C7,L7]
E4.6 -> E1.4 [C6,L6]
E5.7 -> E1.5 [C4,C7,L4,L7]
E5.6 -> E1.4 [C3,C6,L3,L6]
E7.7 -> E8.6 [R1,L7]
E7.6 -> E8.5 [R2,L6]
E8.7 -> E9.4 [C1,C6,L1,L6]
E9.7 -> E9.3 [R1,L7]
E9.6 -> E9.2 [R2,L6]
E10.7 -> E2.5 [R1,R2,C6,L7], E3.5 [R2,C6,C7,L7], E4.5 [C4,C7,L4,L7]
E10.6 -> E2.4 [R2,R3,C5,L6], E3.4 [R3,C5,C6,L6], E4.4 [C3,C6,L3,L6]

Unique Flip Traces:
1: [C1,C7,L1,L7]
2: [x,C6,L6]
3: [C5,C6,L5,L6]
4: [x,C5,C6,L5,L6]
5: [C7,L7]
6: [C6,L6]
7: [C4,C7,L4,L7]
8: [C3,C6,L3,L6]
9: [R1,L7]
10: [R2,L6]
11: [C1,C6,L1,L6]
12: [R1,R2,C6,L7]
13: [R2,C6,C7,L7]
14: [R2,R3,C5,L6]
15: [R3,C5,C6,L6]

**Set Cover Deduction Algorithm:**
1. **Mandatory Bases:** Any feature in a 1-bit flip trace is an absolute
    constraint and is immediately locked.
2. **Greedy Branching:** For remaining traces, iteratively select the feature
    with the highest intersection frequency.
```

```
3. **Collision Verification & Backtracking:** When all traces are covered,
    generate a Truth Table across all 64 rows. If identical inputs yield
    different outputs (a collision), the set is invalid. We backtrack and try
    the next candidate.

<deduction>
Bases: {}
Uncovered: 1,2,3,4,5,6,7,8,9,10,11,12,13,14,15

Freqs: C6:9 L6:9 L7:6 C7:4 C5:4
Attempting: C6
Bases: {C6}
Uncovered: 1,5,7,9,10,14

Freqs: L7:4 C7:3 R2:2 L6:2 C1:1
Attempting: L7
Bases: {C6,L7}
Uncovered: 10,14

Freqs: R2:2 L6:2 R3:1 C5:1
Attempting: R2
Bases: {C6,L7,R2}
Uncovered: None

All traces covered. Validating against 64-row global table:

Test TT for [C6,L7,R2]:
|C6|L7|R2||Out|Ref|
|---|---|---||---|---|
|0|0|0||1|E1.5,E1.4,...|
|0|1|0||0|E1.7,E4.7,...|
|1|0|0||0|E1.6,E2.6,...|
|1|0|1||1|E1.1,E1.0,...|
|1|1|0||1|E2.7,E3.7|
No collisions, but sub-optimal to target bounds. Backtracking... (BT #1)
Removing R2. Backtrack #2.

Freqs: R2:2 L6:2 R3:1 C5:1
Attempting: L6
Bases: {C6,L7,L6}
Uncovered: None

All traces covered. Validating against 64-row global table:

Test TT for [C6,L7,L6]:
|C6|L7|L6||Out|Ref|
|---|---|---||---|---|
|0|0|0||1|E1.5,E1.4,...|
|0|1|0||0|E1.7,E4.7,...|
|1|0|0||1|E1.1,E1.0,...|
|1|0|1||0|E1.6,E2.6,...|
|1|1|1||1|E2.7,E3.7|
No collisions, but sub-optimal to target bounds. Backtracking... (BT #3)
Removing L6. Backtrack #4.
Removing L7. Backtrack #5.

Freqs: L7:4 C7:3 R2:2 L6:2 C1:1
Attempting: C7
Bases: {C6,C7}
Uncovered: 9,10,14

Freqs: R2:2 L6:2 R1:1 L7:1 R3:1
Attempting: R2
Bases: {C6,C7,R2}
Uncovered: 9
```

```
Limit K=3 reached without covering all traces. Backtracking...
Removing R2. Backtrack #6.

Freqs: R2:2 L6:2 R1:1 L7:1 R3:1
Attempting: L6
Bases: {C6,C7,L6}
Uncovered: 9

Limit K=3 reached without covering all traces. Backtracking...
Removing L6. Backtrack #7.

Freqs: R2:2 L6:2 R1:1 L7:1 R3:1
Attempting: L7
Bases: {C6,C7,L7}
Uncovered: 10,14

Limit K=3 reached without covering all traces. Backtracking...
Removing L7. Backtrack #8.
Removing C7. Backtrack #9.

Freqs: L7:4 C7:3 R2:2 L6:2 C1:1
Attempting: L6
Bases: {C6,L6}
Uncovered: 1,5,7,9

Freqs: L7:4 C7:3 C1:1 L1:1 C4:1
Attempting: L7
Bases: {C6,L6,L7}
Uncovered: None

All traces covered. Validating against 64-row global table:

Test TT for [C6,L6,L7]:
|C6|L6|L7||Out|Ref|
|---|---|---||---|---|
|0|0|0||1|E1.5,E1.4,...|
|0|0|1||0|E1.7,E4.7,...|
|1|0|0||1|E1.1,E1.0,...|
|1|1|0||0|E1.6,E2.6,...|
|1|1|1||1|E2.7,E3.7|
No collisions, but sub-optimal to target bounds. Backtracking... (BT #10)
Removing L7. Backtrack #11.

Freqs: L7:4 C7:3 C1:1 L1:1 C4:1
Attempting: C7
Bases: {C6,L6,C7}
Uncovered: 9

Limit K=3 reached without covering all traces. Backtracking...
Removing C7. Backtrack #12.
Removing L6. Backtrack #13.
Removing C6. Backtrack #14.

Freqs: C6:9 L6:9 L7:6 C7:4 C5:4
Attempting: L6
Bases: {L6}
Uncovered: 1,5,7,9,12,13

Freqs: L7:6 C7:4 R1:2 R2:2 C6:2
Attempting: L7
Bases: {L6,L7}
Uncovered: None

All traces covered. Validating against 64-row global table:
```

```
Test TT for [L6,L7]:
|L6|L7||Out|Ref|
|---|---||---|---|
|0|0||1|E1.5,E1.4,...|
|0|1||0|E1.7,E4.7,...|
|1|0||0|E1.6,E2.6,...|
|1|1||1|E2.7,E3.7|
No collisions. 64-row check passed.

Conclusion: Set successfully covered and verified against full data.
Final Bases: [L6,L7]
</deduction>

Final Truth Table:
|L6|L7||Out|
|---|---||---|
|0|0||1|
|0|1||0|
|1|0||0|
|1|1||1|

Target:
Input: 11000010
Bases: [L6,L7]
L6: 10000000
L7: 00000000
Eval:
Bit 7: (1,0) -> 0
Bit 6: (0,0) -> 1
Bit 5: (0,0) -> 1
Bit 4: (0,0) -> 1
Bit 3: (0,0) -> 1
Bit 2: (0,0) -> 1
Bit 1: (0,0) -> 1
Bit 0: (0,0) -> 1

<output>
\boxed{01111111}
</output>
```